

Identifying Player Roles From Event Data

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse
from matplotlib.patches import Rectangle
from mplsoccer import Pitch
import seaborn as sns

from position_identification_alg import *
from visualization_tool import *
```

Loading statsbomb event data

```
In [2]: from statsbombpy import sb

sb.competitions().tail(40)
sb.matches(2,27)
events = sb.events(3754058)
```

We will use the AFC Bournemouth vs. Leicester City game from Premier League 2015/16

```
In [3]: events["x"] = events["location"].str[0]
events["y"] = events["location"].str[1]
```

Here we will focus on Leicester City and identify player roles from their players' events.

We can identify their overall role using all the events they perform. However, we can also find in possession and out of possession roles as well.

To identify in possession roles we utilize events that occur when the team has the ball.

To identify out of possession roles we utilize events that occur when the team does not have the ball.

```
In [4]: team_match_events = events[events.team_id==22]

starting_lineup = team_match_events[team_match_events.type=="Starting XI"].iloc[0].
players = [player["player"]["id"] for player in starting_lineup if player["position"]
team_subs = team_match_events[team_match_events.type == "Substitution"]

valid_events = ['Pass', 'Ball Receipt*', 'Carry', 'Pressure', 'Dispossessed', 'Duel',
                'Ball Recovery', 'Miscontrol', 'Clearance', 'Dribbled Past', 'Dribble', 'Inte

in_possession_events = ['Pass', 'Ball Receipt*', 'Carry', 'Dispossessed', 'Shot',

out_possession_events = ['Pressure', 'Duel', 'Block', 'Ball Recovery', 'Dribbled Past
```

```
valid_team_match_events = team_match_events[team_match_events.type.isin(valid_event)]
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
```

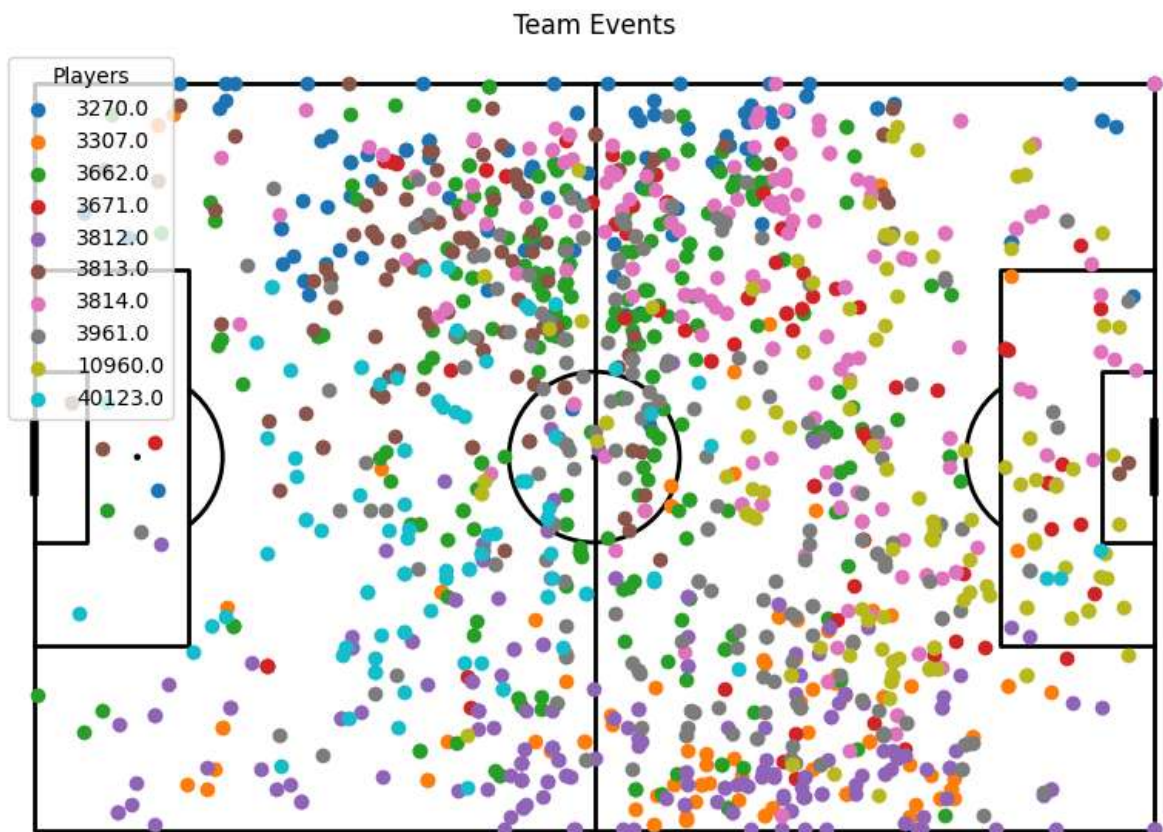
```
In [5]: valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
```

Events for whole team

Below we show all the events performed by the starting 10 field players. At first it may seem daunting how we can identify roles from these events but I will break it down over the next few sections.

```
In [6]: plot_team_events(valid_team_match_events)
```

```
Out[6]: (<Figure size 1000x700 with 1 Axes>, <Axes: title={'center': 'Team Events'}>)
```

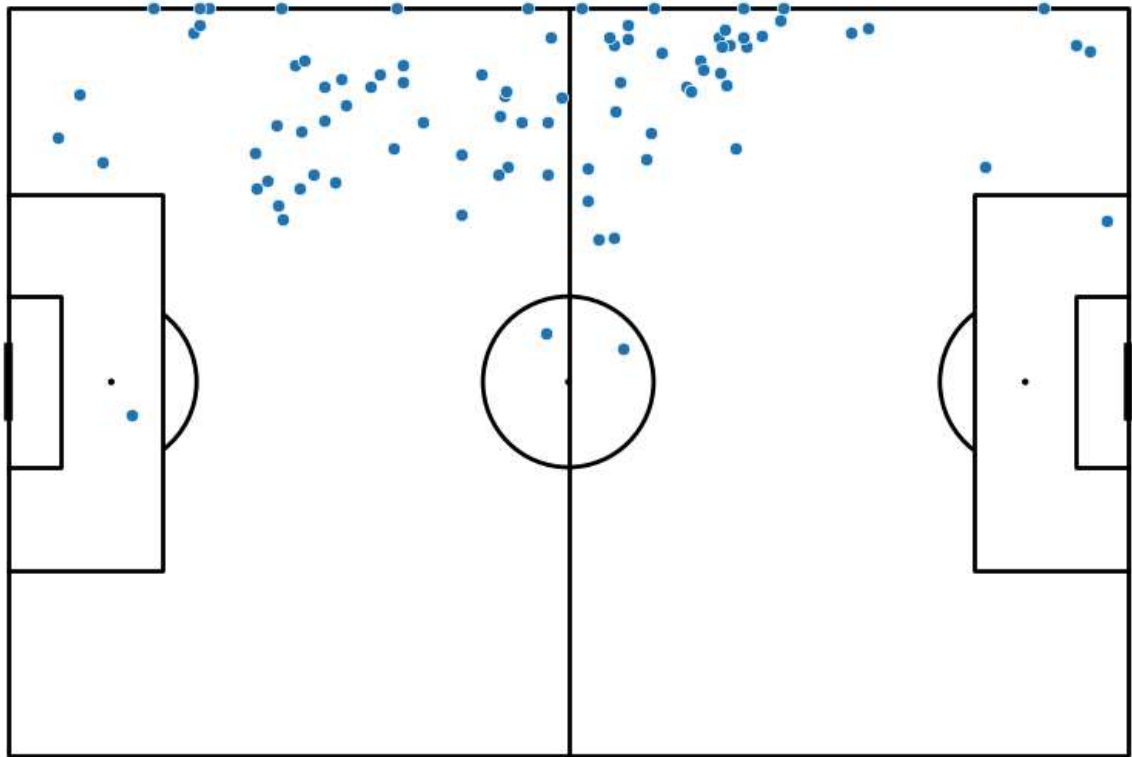


Events for a player

First step is to break down the team's events into each individual players events.

```
In [7]: player_match_events = valid_team_match_events[valid_team_match_events.player_id==32]
plot_player_events(player_match_events)
```

```
Out[7]: (<Figure size 1000x700 with 1 Axes>, <Axes: xlabel='x', ylabel='y'>)
```



Estimating gaussian distribution of events for player.

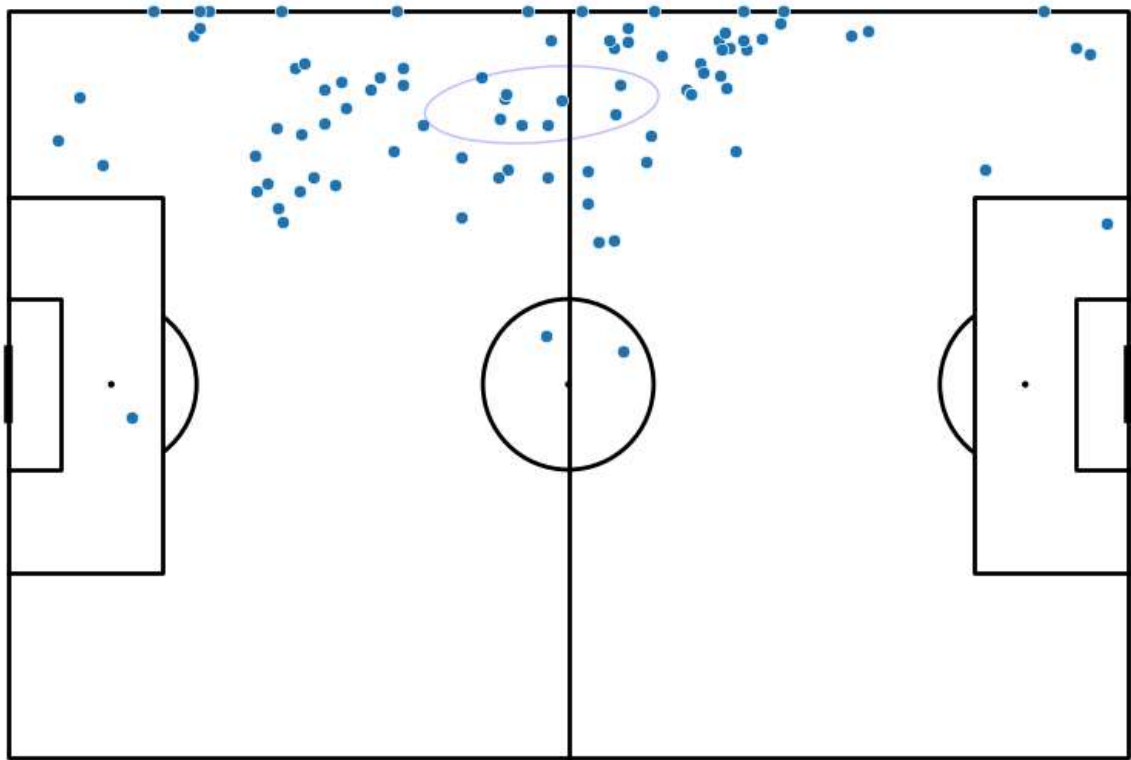
Next step is to create a gaussian distribution to represent the events the player performs. This now gives us a model that shows the general positioning tendencies of the player throughout the game. These gaussians give us both a central tendency as well as variance in positioning these will be crucial for identifying roles later.

```
In [8]: mu, Sigma = create_player_gaussian(player_match_events)
```

Here I show the gaussian model at .5 standard deviations.

```
In [9]: plot_player_gaussian(player_match_events,mu,Sigma)
```

```
Out[9]: (<Figure size 1000x700 with 1 Axes>, <Axes: xlabel='x', ylabel='y'>)
```



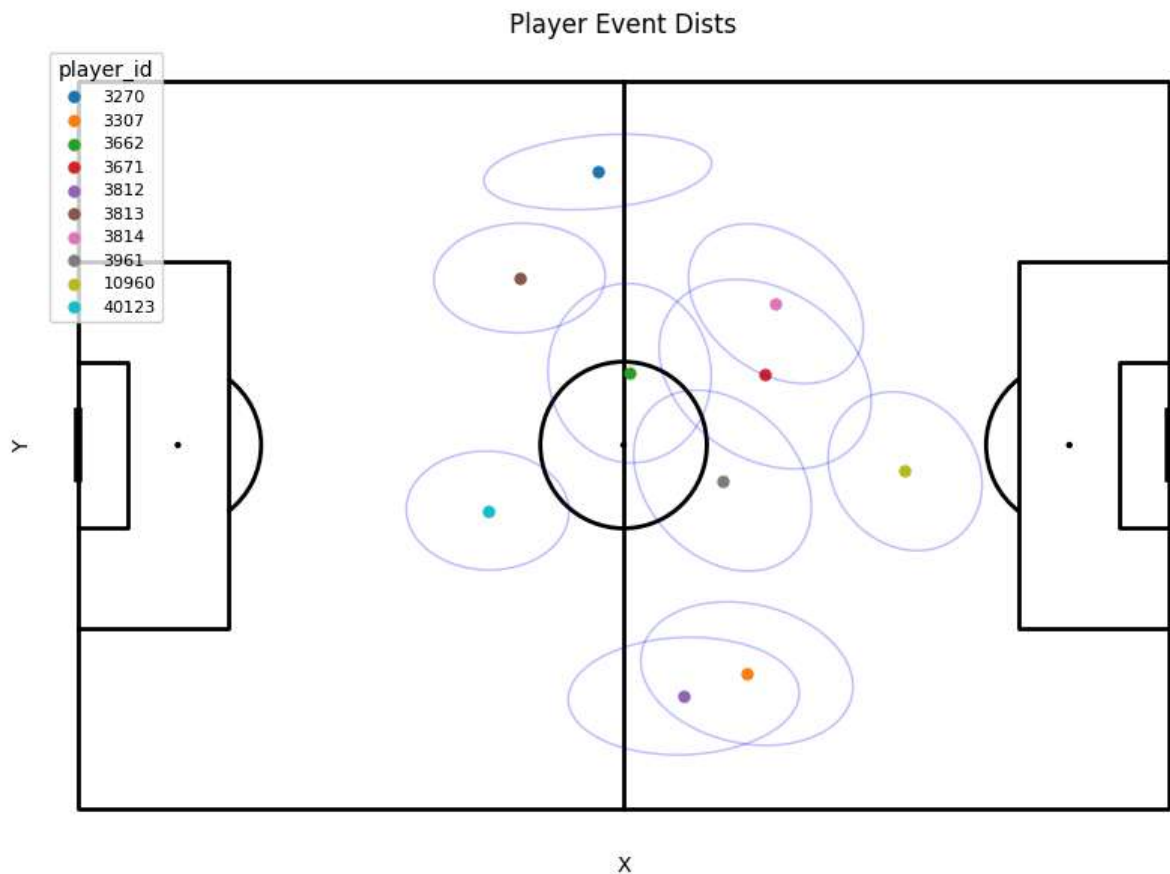
Gaussian Distributions of the Team

Now we can compute the gaussian models for each and every player. We can visualize the central tendency and variance of each of the player's positioning by events throughout the game

```
In [10]: player_dists, zone_probs_df, player_pos = position_calculations(valid_team_match_ev
```

```
In [11]: plot_team_player_gaussians(player_dists)
```

```
Out[11]: (<Figure size 1000x700 with 1 Axes>,
  <Axes: title={'center': 'Player Event Dists'}, xlabel='X', ylabel='Y'>)
```



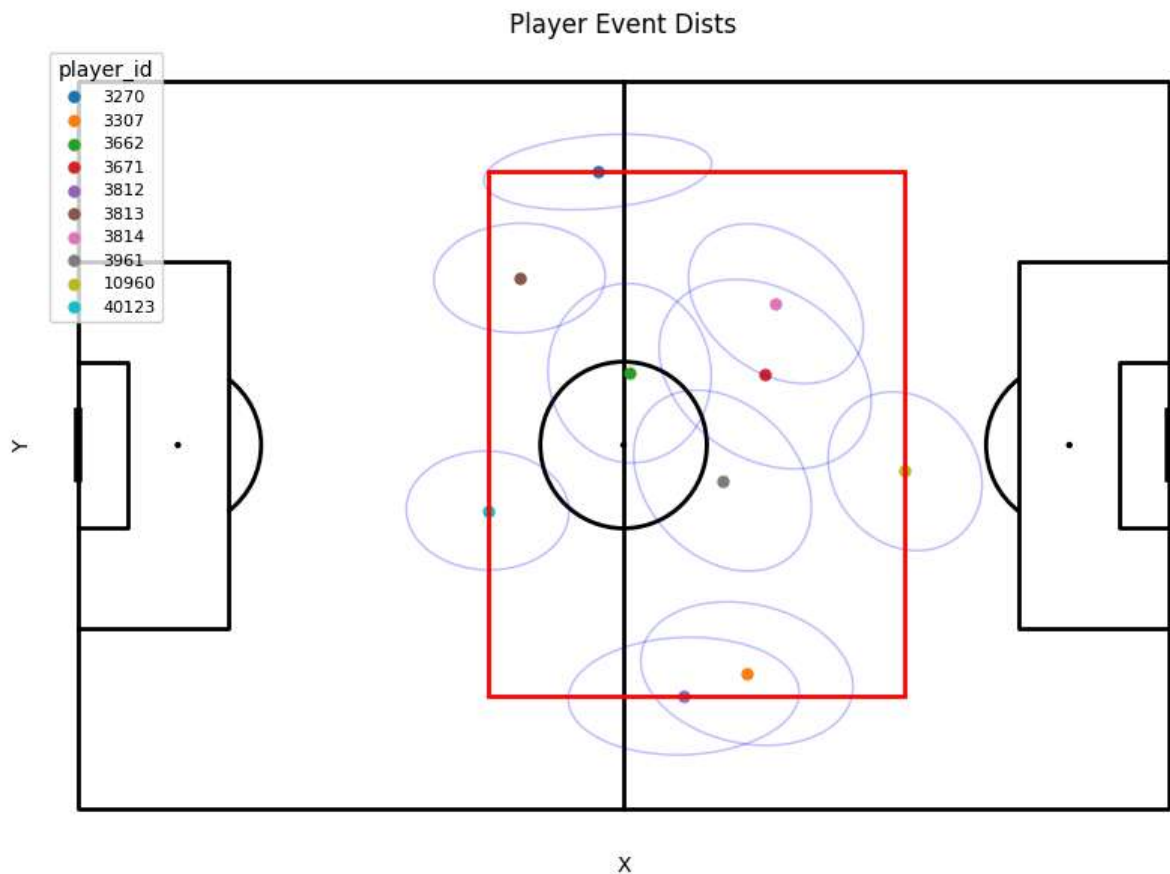
We can see that some of the player's roles are beginning to appear evident. But how can we actually assign the player's a role?

Team Bounding

We can now create a box that tightly surrounds the team. This box allows us to anchor the player's position compared to the team's general positioning.

```
In [12]: plot_team_player_gaussians(player_dists,with_box=True)
```

```
Out[12]: (<Figure size 1000x700 with 1 Axes>,
<Axes: title={'center': 'Player Event Dists'}, xlabel='X', ylabel='Y'>)
```

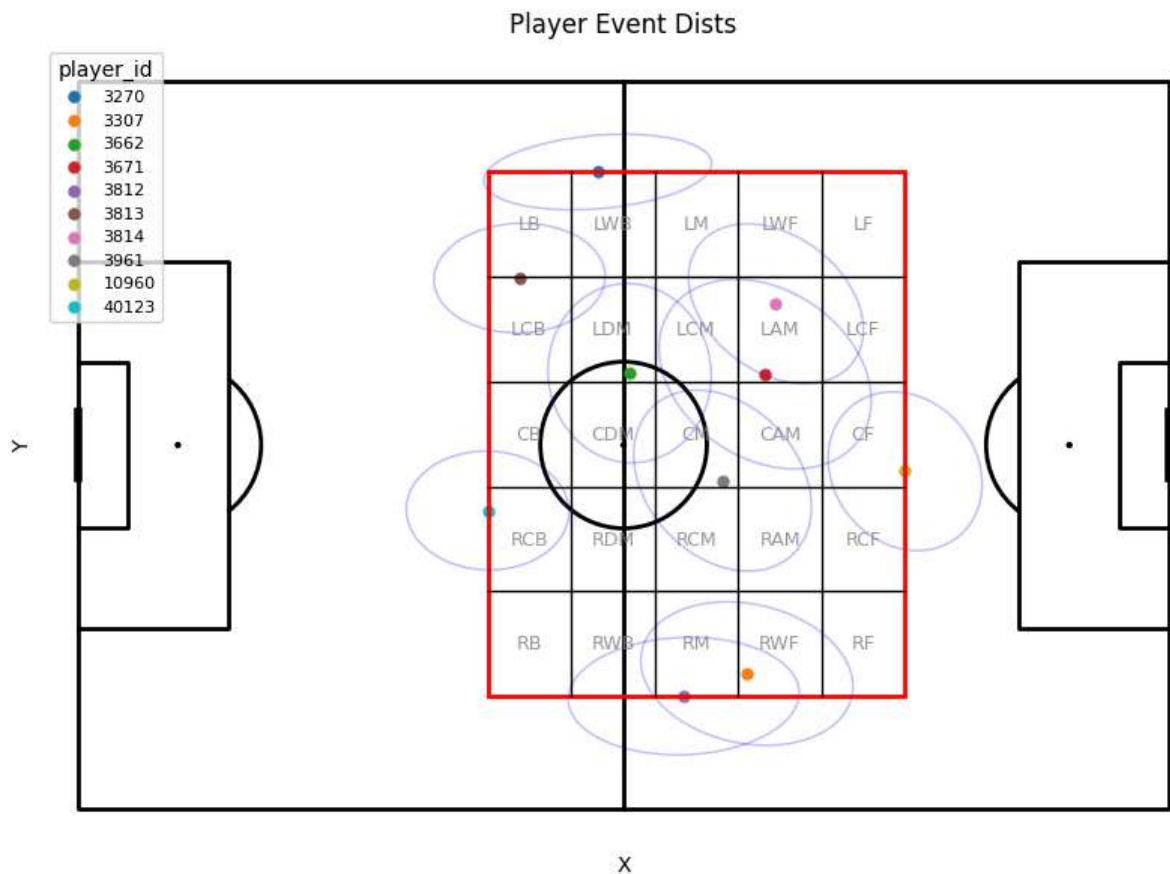


Role assignments within box.

We can split the box into 5x5 cells where each cell represents a role. From left to right we represent our back to forward roles and from top to bottom we represent our left side to right side roles.

```
In [13]: plot_team_player_gaussians(player_dists,with_box=True,with_cells=True,nx=5,ny=5)
```

```
Out[13]: (<Figure size 1000x700 with 1 Axes>,
<Axes: title={'center': 'Player Event Dists'}, xlabel='X', ylabel='Y'>)
```



Calculate role probabilities

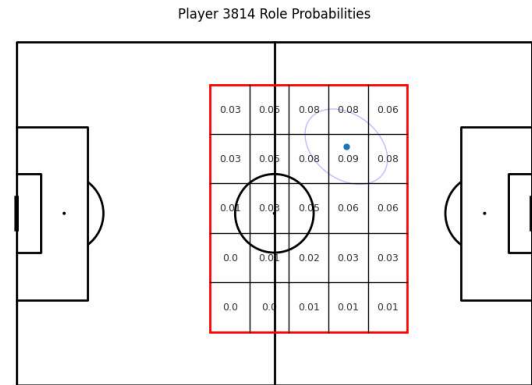
We can now calculate the probability the player falls into each of the 25 roles. We do this by utilizing the gaussian model that represents the player's positioning and integrating over each zone to compute the probability the player fits into that role.

```
In [14]: fig, axes = plt.subplots(1, 2, figsize=(20,10))

plot_role_assignments(ax=axes[0])
plot_player_role_probabilities(player_dists,zone_probs_df,3814,ax=axes[1])
```

```
Out[14]: (<Figure size 2000x1000 with 2 Axes>,
<Axes: title={'center': 'Player 3814 Role Probabilities'}>)
```


Player Position/Role Assignment				
LB	LWB	LM	LWF	LF
LCB	LDM	LCM	LAM	LCF
CB	CDM	CM	CAM	CF
RCB	RDM	RCM	RAM	RCF
RB	RWB	RM	RWF	RF



Find Each Player's Probable Role

We can now find the role that has the highest probability for the player and assign them that role. We can do that for each player.

```
In [15]: fig, axes = plt.subplots(1, 2, figsize=(20,10))

plot_role_assignments(ax=axes[0])

plot_player_probable_position(player_pos,team_subs,use_subbing=False,ax=axes[1])
```

```
Out[15]: (<Figure size 2000x1000 with 2 Axes>,
<Axes: title={'center': 'Players by Most Probable Role'}>)
```

Player Position/Role Assignment				
LB	LWB	LM	LWF	LF
LCB	LDM	LCM	LAM	LCF
CB	CDM	CM	CAM	CF
RCB	RDM	RCM	RAM	RCF
RB	RWB	RM	RWF	RF

Players by Most Probable Role				
3270				
3813	3662		3671	
			3814	
		3961		10960
40123				
		3812	3307	

Green: Sub On Red: Sub Off Orange: Sub On+Off

We can also compare how the player's role changes in possession vs. out of possession

```
In [16]: players = [player["player"]["id"] for player in starting_lineup if player["position"]

valid_team_match_events = team_match_events[team_match_events.type.isin(in_possession)]
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
player_dists_in, zone_probs_df_in, player_pos_in = position_calculations(valid_team

fig, axes = plt.subplots(1, 2, figsize=(20,10))

plot_team_events(valid_team_match_events, ax=axes[0])
axes[0].set_title("In Possession - Events")

plot_team_player_gaussians(
    player_dists_in,
    with_box=True,
    with_cells=True,
    ax=axes[1]
)
axes[1].set_title("In Possession - Player Roles")
plt.show()

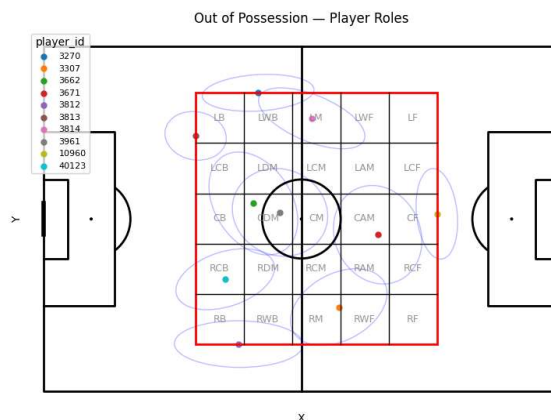
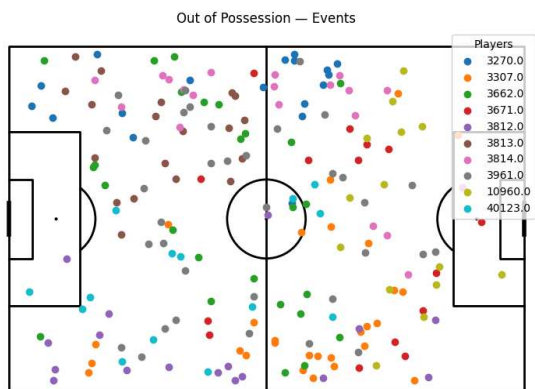
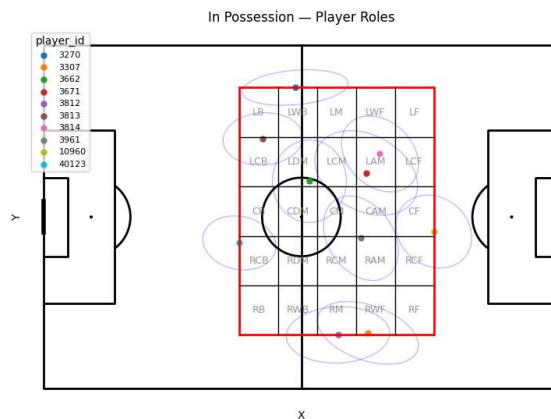
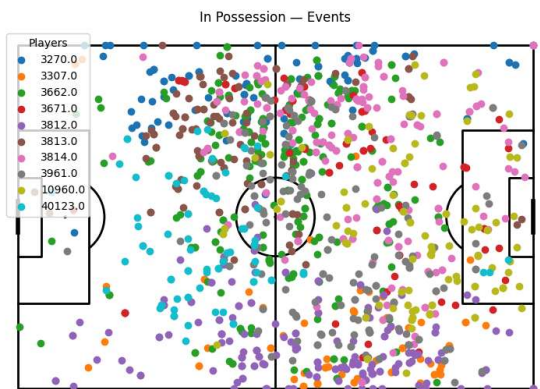
valid_team_match_events = team_match_events[team_match_events.type.isin(out_possession)]
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
player_dists_out, zone_probs_df_out, player_pos_out = position_calculations(valid_t

fig, axes = plt.subplots(1, 2, figsize=(20,10))

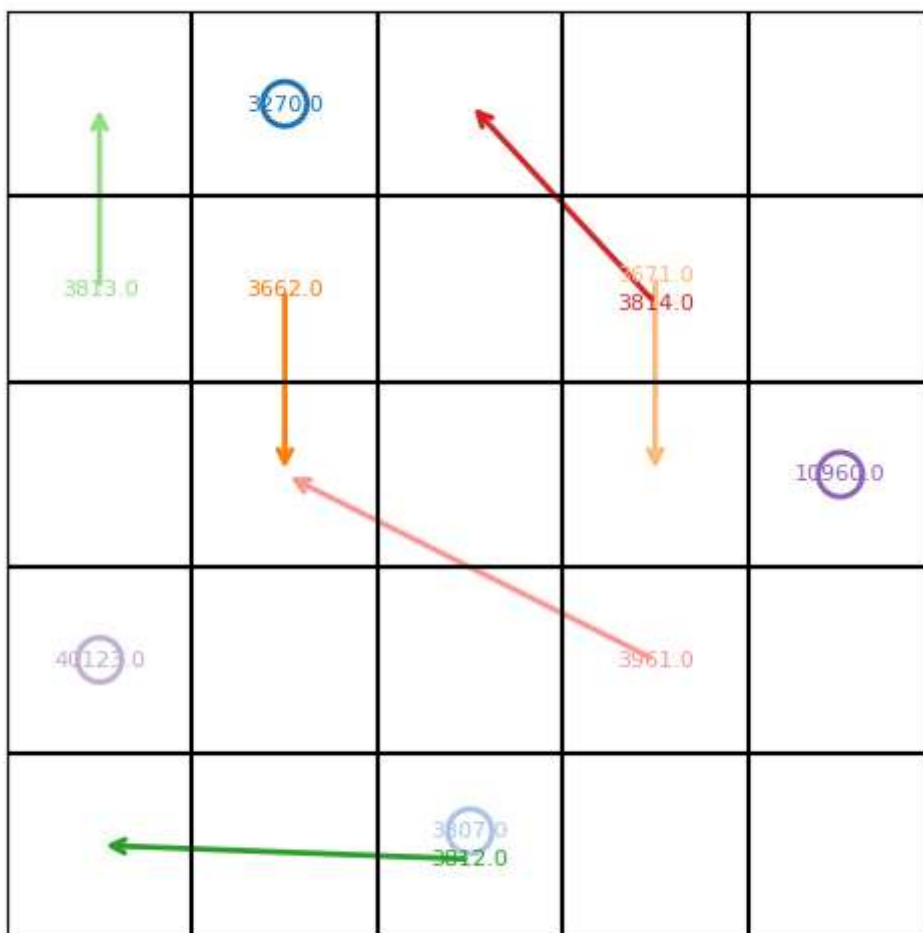
plot_team_events(valid_team_match_events, ax=axes[0])
axes[0].set_title("Out of Possession - Events")

plot_team_player_gaussians(
    player_dists_out,
    with_box=True,
    with_cells=True,
    ax=axes[1]
)
axes[1].set_title("Out of Possession - Player Roles")
plt.show()

plot_role_transition_arrows(player_pos_in=player_pos_in,player_pos_out=player_pos_o
plt.show()
```



In → Out of Possession Role Transitions



By breaking down the role into in possession and out of possession we are now able to identify key characteristics and play styles of players that are otherwise not clear in traditional "position" labels.

How can we deal with substitutions?

The same 10 field players are not on the field, so how can we take this into consideration when assigning player roles. One approach to this problem is to break the events up between substitutions and to calculate player role probabilities between each sub. We can then sum up all the broken player role maps to create a final role probability map for a player.

This allows the players role to be flexible to the players that are currently on the field. However, not all segments are equal some subs occur in close proximity so there is not a lot of events that occur in the short segment. We utilize weighting by the segment duration to take this into consideration and not heavily impact the player's final role assignment.

```
In [17]: players = [player["player"]["id"] for player in starting_lineup if player["position"]

valid_team_match_events = team_match_events[team_match_events.type.isin(valid_event
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
first_sub = team_subs.iloc[0]
valid_team_match_events = valid_team_match_events[(valid_team_match_events.minute<=
valid_team_match_events = valid_team_match_events[valid_team_match_events.player_id
duration = first_sub.minute
player_dists, zone_probs_df, player_pos = position_calculations(valid_team_match_ev
zone_probs_df["duration"] = duration

print("Start of game")
print("Duration:", duration)

fig, axes = plt.subplots(1, 2, figsize=(20,10))

plot_team_events(valid_team_match_events, ax=axes[0])
axes[0].set_title("Team Events")

plot_team_player_gaussians(
    player_dists,
    with_box=True,
    with_cells=True,
    ax=axes[1]
)
axes[1].set_title("Player Roles")
plt.show()

for i in range(team_subs.shape[0]):
    print(f"After sub {i+1}")

    valid_team_match_events = team_match_events[team_match_events.type.isin(valid_e
```

```

valid_team_match_events = valid_team_match_events[valid_team_match_events.playe

sub = team_subs.iloc[i]
player_in = int(sub["substitution_replacement_id"])
player_out = int(sub["player_id"])

period = sub["period"]
minute = sub["minute"]
players.remove(player_out)
players.append(player_in)

if i+1 < team_subs.shape[0]:
    next_sub = team_subs.iloc[i+1]
    next_period = next_sub["period"]
    next_minute = next_sub["minute"]
else:
    max_minute = team_match_events["minute"].max()
    next_period = 2
    next_minute = max_minute

duration = next_minute-minute
print("Duration:", duration)

valid_team_match_events = valid_team_match_events[(valid_team_match_events.peri
valid_team_match_events = valid_team_match_events[valid_team_match_events.playe

player_dists_sub, zone_probs_df_sub, player_pos_sub = position_calculations(val

zone_probs_df_sub["duration"] = duration

zone_probs_df = pd.concat([zone_probs_df, zone_probs_df_sub])

fig, axes = plt.subplots(1, 2, figsize=(20,10))

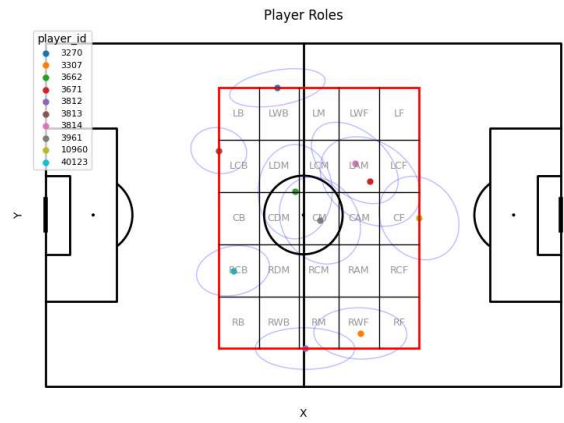
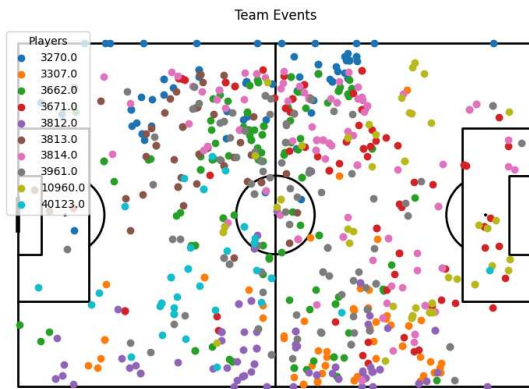
plot_team_events(valid_team_match_events, ax=axes[0])
axes[0].set_title("Team Events")

plot_team_player_gaussians(
    player_dists_sub,
    with_box=True,
    with_cells=True,
    ax=axes[1]
)
axes[1].set_title("Player Roles")
plt.show()

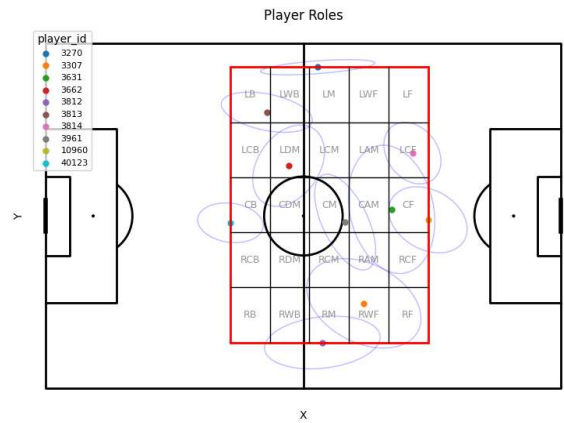
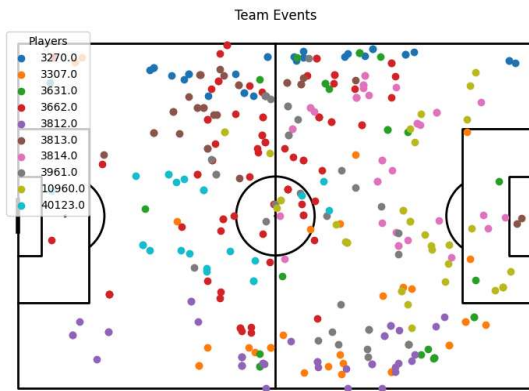
```

Start of game

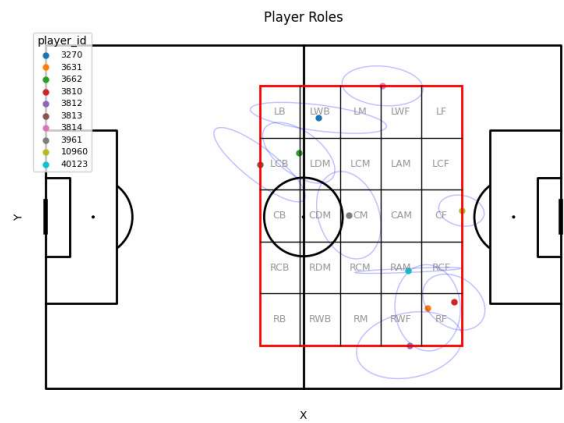
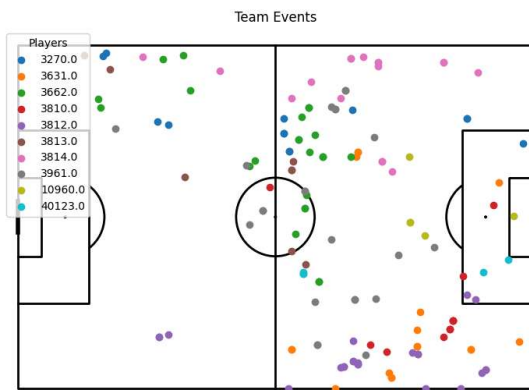
Duration: 45



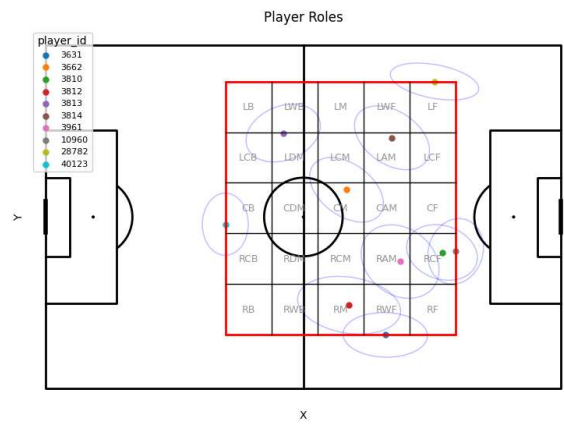
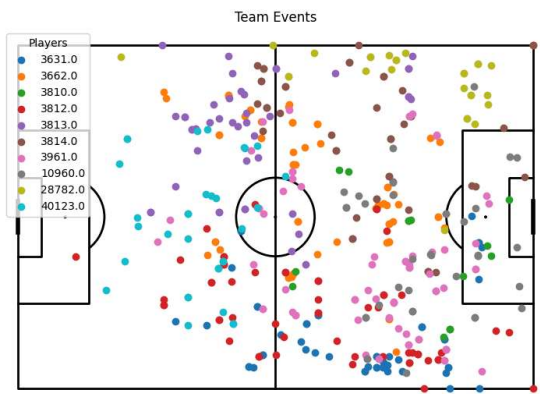
After sub 1
Duration: 19



After sub 2
Duration: 9



After sub 3
Duration: 21



We compute the player role probabilities for each segment below

In [18]: zone_probs_df

Out[18]:

	RB	RWB	RM	RWF	RF	RCB	R
player_id							
3270.0	3.028328e-09	1.233758e-09	4.116085e-10	1.124403e-10	2.514757e-11	5.788997e-06	2.97808
40123.0	9.539679e-02	7.216135e-02	4.038491e-02	1.671916e-02	5.119170e-03	1.586358e-01	1.30785
3961.0	1.699023e-02	2.733945e-02	3.439046e-02	3.381872e-02	2.599841e-02	3.177588e-02	4.85784
3662.0	2.104168e-02	2.717588e-02	2.639195e-02	1.927263e-02	1.058192e-02	3.820291e-02	4.93945
3812.0	1.495697e-01	1.793620e-01	1.832837e-01	1.595966e-01	1.184209e-01	3.782459e-02	4.50918
10960.0	2.618386e-03	7.895522e-03	1.851148e-02	3.374923e-02	4.785034e-02	5.836491e-03	1.67590
3307.0	6.770843e-02	1.072098e-01	1.414468e-01	1.554983e-01	1.424406e-01	3.608677e-02	5.63706
3671.0	4.498209e-03	8.646827e-03	1.401670e-02	1.916093e-02	2.208900e-02	1.227430e-02	2.20755
3813.0	4.544714e-04	3.473990e-04	1.614735e-04	4.560000e-05	7.813965e-06	1.207546e-02	8.59940
3814.0	8.396596e-04	2.449973e-03	5.533233e-03	9.674401e-03	1.309630e-02	4.471746e-03	1.12294
3814.0	1.286397e-05	1.216637e-04	7.010041e-04	2.464549e-03	5.293798e-03	1.764197e-04	1.41884
10960.0	1.262527e-03	4.542119e-03	1.248945e-02	2.625266e-02	4.219009e-02	5.171992e-03	1.65050
3662.0	1.811026e-02	1.478616e-02	8.662755e-03	3.641167e-03	1.097693e-03	4.753546e-02	4.43142
3813.0	2.252984e-07	5.449919e-07	1.060765e-06	1.661514e-06	2.094586e-06	1.329336e-04	2.48124
40123.0	2.408807e-02	2.001768e-02	1.163081e-02	4.723931e-03	1.340763e-03	1.661497e-01	1.24420
3961.0	2.345719e-03	1.234640e-02	3.555539e-02	5.614888e-02	4.867771e-02	8.628193e-03	3.33915
3270.0	1.523172e-53	-4.078841e-54	-6.726047e-54	2.774576e-55	2.263533e-54	-8.234530e-36	-6.04445
3812.0	1.219633e-01	1.479370e-01	1.591427e-01	1.518310e-01	1.284691e-01	3.822050e-02	4.96398
3307.0	4.604284e-02	6.431656e-02	7.861410e-02	8.408088e-02	7.868900e-02	5.128459e-02	6.61994

	RB	RWB	RM	RWF	RF	RCB	R
player_id							
3631.0	9.589777e-03	1.983984e-02	3.318190e-02	4.486586e-02	4.904487e-02	1.432710e-02	2.85442
3814.0	6.910832e-09	1.676140e-08	3.171809e-08	4.683369e-08	5.396200e-08	4.632062e-06	1.02840
3961.0	1.533342e-02	2.985852e-02	3.884596e-02	3.377082e-02	1.961648e-02	3.080559e-02	5.48571
3812.0	6.281816e-02	9.747638e-02	1.300768e-01	1.492762e-01	1.473241e-01	2.836013e-02	4.68182
3270.0	6.085722e-13	2.358823e-12	8.089028e-12	2.454360e-11	6.589391e-11	1.053255e-07	2.83037
3662.0	4.945335e-04	1.520550e-03	3.023746e-03	3.893515e-03	3.249124e-03	8.264187e-03	1.77451
3631.0	1.027041e-02	3.303714e-02	7.348076e-02	1.130448e-01	1.203142e-01	9.698153e-03	3.08508
3810.0	2.667300e-03	1.490619e-02	5.372821e-02	1.250156e-01	1.879003e-01	4.620109e-03	2.19685
40123.0	8.608621e-07	2.018339e-07	3.545801e-08	4.663156e-09	4.586833e-10	1.197433e-01	1.75715
3813.0	8.557768e-04	4.529705e-03	1.314047e-02	2.107980e-02	1.884102e-02	2.078536e-02	5.07472
10960.0	1.693714e-07	5.477173e-06	8.162028e-05	5.641957e-04	1.820143e-03	1.901340e-05	4.94288
3961.0	8.786284e-03	2.595278e-02	5.364439e-02	7.761612e-02	7.862002e-02	1.468501e-02	3.96047
3812.0	7.149784e-02	1.016128e-01	1.177048e-01	1.111311e-01	8.552051e-02	6.189473e-02	8.14847
3813.0	1.680926e-03	1.171831e-03	5.493352e-04	1.730900e-04	3.663514e-05	1.503342e-02	1.19828
3814.0	3.399198e-05	2.087346e-04	8.347719e-04	2.176306e-03	3.701530e-03	4.874526e-04	2.43057
3662.0	1.583362e-03	5.674489e-03	1.291426e-02	1.867954e-02	1.717973e-02	9.491611e-03	2.70590
40123.0	9.316968e-02	3.689253e-02	5.758597e-03	3.504256e-04	8.212712e-06	1.960334e-01	7.77302
3631.0	5.866685e-02	1.194136e-01	1.816074e-01	2.063817e-01	1.752585e-01	1.950507e-02	3.89263
28782.0	5.555626e-13	6.027700e-12	4.793888e-11	2.795986e-10	1.196398e-09	5.587407e-09	4.46011

	RB	RWB	RM	RWF	RF	RCB	R
player_id							
10960.0	2.748578e-04	3.563053e-03	2.374806e-02	8.169601e-02	1.454866e-01	2.908991e-04	4.04735
3810.0	1.464955e-03	7.910051e-03	2.792326e-02	6.449475e-02	9.752072e-02	3.729024e-03	1.81556

40 rows × 26 columns

Then we can perform our weighted average (by duration) to find the final role probability map

```
In [19]: def weighted_positioning(df):
zone_cols = [c for c in df.columns if c != "duration"]

total_min = df["duration"].sum()

weights = df["duration"] / total_min
return (df[zone_cols].multiply(weights, axis=0)).sum()

role_probs = zone_probs_df.groupby("player_id").apply(weighted_positioning)
```

```
In [20]: role_probs
```

Out[20]:

	RB	RWB	RM	RWF	RF	RCB	RDM
player_id							
3270.0	1.866853e-09	7.608268e-10	2.547285e-10	7.233842e-11	2.362583e-11	3.581545e-06	1.870699e-06
3307.0	6.127646e-02	9.447584e-02	1.227934e-01	1.342962e-01	1.235143e-01	4.059862e-02	5.928858e-02
3631.0	3.074782e-02	6.493831e-02	1.041947e-01	1.266096e-01	1.162267e-01	1.569602e-02	3.341735e-02
3662.0	1.413481e-02	1.741171e-02	1.756003e-02	1.450811e-02	9.436798e-03	3.080858e-02	4.034744e-02
3671.0	4.498209e-03	8.646827e-03	1.401670e-02	1.916093e-02	2.208900e-02	1.227430e-02	2.207593e-02
3810.0	1.825659e-03	1.000889e-02	3.566475e-02	8.265102e-02	1.246346e-01	3.996350e-03	1.929954e-02
3812.0	1.182421e-01	1.478005e-01	1.586592e-01	1.462115e-01	1.158691e-01	4.237581e-02	5.430674e-02
3813.0	6.750736e-04	8.619054e-04	1.458370e-03	2.079113e-03	1.816276e-03	1.115629e-02	1.170269e-02
3814.0	4.121595e-04	1.244085e-03	2.977075e-03	5.615716e-03	8.166472e-03	2.285731e-03	6.206588e-03
3961.0	1.203874e-02	2.424034e-02	3.935393e-02	4.811220e-02	4.172738e-02	2.318602e-02	4.410520e-02
10960.0	1.570095e-03	5.494385e-03	1.669957e-02	3.976819e-02	6.411146e-02	3.906276e-03	1.231060e-02
28782.0	5.555626e-13	6.027700e-12	4.793888e-11	2.795986e-10	1.196398e-09	5.587407e-09	4.460115e-08
40123.0	7.135213e-02	4.683342e-02	2.297061e-02	9.036976e-03	2.723507e-03	1.647856e-01	1.219467e-01

13 rows × 25 columns



Then we can find the most probable role to assign to each player

```
In [21]: player_roles = role_probs.idxmax(axis=1)
```

```
In [22]: fig, axes = plt.subplots(1, 2, figsize=(20,10))

plot_role_assignments(ax=axes[0])
plot_player_probable_position(player_roles,team_subs,ax=axes[1])
```

```
Out[22]: (<Figure size 2000x1000 with 2 Axes>,
<Axes: title={'center': 'Players by Most Probable Role'}>)
```

Player Position/Role Assignment				
LB	LWB	LM	LWF	LF
LCB	LDM	LCM	LAM	LCF
CB	CDM	CM	CAM	CF
RCB	RDM	RCM	RAM	RCF
RB	RWB	RM	RWF	RF

Players by Most Probable Role				
3270				28782
3813	3662		3671	3814
		3961		10960
40123				3810
		3812	3307	3631

Green: Sub On Red: Sub Off Orange: Sub On+Off

Conclusion

In this notebook we looked at how we can identify player roles from event data. Here player roles defines the role that a player takes within a teams formation. We utilize gaussian functions to model each of the player's positioning from events. We then bound all the teams gaussians into a box to anchor the player relationships and identify the roles played among players of a team. We use a 5x5 grid within the box with each cell representing a role. We are able to then use integration to find the probability of each player falling into a role. Overall this models the process for how we can assign roles from event data.

We are able to utilize defensive vs. offensive events to assign players different roles for when the team is in possession vs. out of possession. This enables us to learn even more information about the role a player takes when the team is in different scenarios. This also allows us to look at how players transition between phases of play and how their role changes.

To make this approach usable we show a method for how to incorporate this model to work within game substitutions by breaking the game into segments between subs and computing player role probability maps for each segment then using weighted averages to bring together each of the segments into a final role probability map that can be used to assign player roles.

These approaches help to assign and detect player roles without tracking data which is not available everywhere and gives coaches, analysts, and scouts the ability to analyze and compare players based off the role they play from the actions that they perform during the game.